

# Caching and Sharding in Game Development Production Architecture

and Operating Model Date: 2025-08-10 UTC

Caching and Sharding in Game Development Production Architecture  
and Operating Model Date: 2025-08-10 UTC

## One-Pager Summary

Goal Apply caching and sharding techniques to deliver sub-150 ms gameplay APIs, low-jitter realtime loops, and cost-efficient scale for live games. The platform supports session-based action games, MMOs, and live-ops services such as inventory, leaderboards, matchmaking, and cosmetics.

## Key Outcomes

Hot-path latency reduced via multi-tier caches at client, edge, and region. Predictable scale using shard-per-realm or per-feature sharding with automated rebalancing. Graceful degradation under load with cache-first fallbacks and bounded staleness. Operational excellence with SLOs, autoscaling by tail latency, and fast canary rollback.

## Primary Assumptions

Global player base across several regions, concurrent players from one hundred thousand to many millions. Mix of authoritative realtime servers and HTTP or gRPC backends. Web, PC, console, and mobile clients. Data classes: realtime transient state, session state, profile and inventory, leaderboards, world state, content and media.

## Functional Requirements

Realtime Authoritative tick servers with interest management; publish snapshots to clients with prediction and reconciliation on device. Hot services Matchmaking, presence, chat metadata, inventory reads, price list, entitlements, live configs, limited-time events. Cold services Accounts, purchases, audit trails, catalogs, telemetry, analytics. Caching Client and edge caches for static and semi-static content. Regional caches for hot reads and computed views. Write-through and write-behind where safe. Sharding Per-realm or per-world shards for gameplay. Feature shards for profiles, inventories, and leaderboards. Cross-shard operations with bounded fanout. Tooling Admin consoles, runbooks, backfills, replay, and roll-forward workflows.

## Non-Functional Requirements

Availability 99.95 percent read APIs, 99.9 percent write or admin. Realtime data plane 99.9 percent. Latency targets p95: edge API under 80 ms, inventory read under 50 ms from cache, leaderboard top-100 under 120 ms, matchmaking placement under 400 ms. Durability: RPO 0 for purchases, currency, inventory mutations, match outcomes. RTO 15 minutes per region for stateful stores. Compliance: SOC 2 Type 2, GDPR, CCPA with data minimization and residency.

## Architecture Overview

Cell-Based Layout Each region is a cell with Edge and CDN, API and Websocket Gateway, Realtime Game Servers, Feature Services, Caches, Storage, and Observability. A thin global control plane manages identity, config, shard maps, and deploys. Caching Tiers Client cache: in-memory LRU per screen with ETags and TTL hints. Edge cache: CDN or edge workers for static assets, configs, content manifests, precomputed leaderboards. Regional cache: Redis Cluster for hot KV or lists or sorted sets. Optional local in-process caches guarded by version keys. Shard Topologies Gameplay shards: realm or map shards with fixed capacity, authoritative sim per shard. Service shards: profile or inventory sharded by player id hash; leaderboards sharded by board id; chat channels sharded by channel id. Directory A shard map service tracks ownership and routing; lookups cached in clients and gateways with

short TTL and push invalidations.

Data Flow Examples Inventory Read Client → Edge → API Gateway → Cache read. Miss → Inventory service read from OLTP. Write-through populates cache with TTL and version. Inventory Write Client → API Gateway with idempotency key → Inventory service validates and writes to OLTP using conditional updates and outbox to Kafka → Cache invalidation event → Regional caches update. Leaderboard Update Game server emits score to Kafka → Aggregator updates sorted set in Redis and appends to OLAP facts → Periodic snapshot written to object storage and long-term warehouse. Matchmaking Client

pings regions → Gateway picks region and shard based on RTT and capacity → Place into queue →

Assignment published; client connects to realtime shard.

Technology Stack reference Edge and Gateway: Cloud CDN, WAF, anycast DNS, API Gateway, OAuth 2.1 or OIDC, TLS 1.3. Optional QUIC for realtime hints. Realtime: UDP or QUIC data plane, authoritative servers, prediction and reconciliation on clients. Caches: Redis Cluster for regional; CDN edge KV; client LRU. Streaming: Kafka with Schema Registry; Flink for enrich and fanout; Debezium for CDC. OLTP: Postgres or Aurora for correctness domains; DynamoDB or Scylla for high-QPS KV. Search: OpenSearch for text and moderator tools. Analytics: ClickHouse or BigQuery or Snowflake on S3 or GCS with Iceberg or Delta. Platform: Kubernetes with service mesh mTLS and SPIFFE; Argo CD and Rollouts; Terraform or Pulumi.

## Storage Strategy

Hot path Redis Cluster for inventories, leaderboards top windows, presence, configs, matchmaking queues. Partition with hash tags to colocate related keys. Warm path DynamoDB or Scylla for inventory and profile source of truth, conditional writes with version numbers. Postgres for orders, payments, and accounting with outbox. Cold path Object storage for snapshots and telemetry; Parquet with hourly partitions and Iceberg tables. Warehouse for aggregates and ML features. Cache policies Static and semi-static: long TTL with ETag and soft revalidate. Mutable hot KV: short TTL 1 to 5 minutes plus event-driven invalidation. Write-through on read miss for immutable after write objects such as match assignments. Leaderboards: real-time updates to Redis sorted sets; periodic snapshot and truncate tails to control memory.

Sharding Strategy Keys player id hash for profile or inventory; board id for leaderboards; channel id for chat; realm id for gameplay. Routing Directory answers shard for a key; response cached in client and gateway; invalidation on reshard. Gateways maintain sticky sessions to reduce churn. Rebalancing Move partitions by range or vnode. Use dual-write and shadow-read windows. Backfill via Kafka snapshots. Pause writes to a moving range for seconds; clients retry. Cross-Shard Operations Fanout queries bounded by small K; batch read with scatter gather; write orchestration via saga when a transaction spans shards.

## Data Models

abridged Profile player id, tenant id, level, cosmetics, currency balances, updated at, version. InventoryItem player id, item id, quantity, attributes json, acquired at, expires at optional, version. LeaderboardEntry board id, player id, score, rank snapshot, updated at. ShardRecord shard id, type realm or feature, region, owners, capacity, load metrics, state, version. CacheEntry key, value checksum or version, ttl, last refresh, invalidation token. AuditEvent actor, action, target, attributes summary, timestamp, trace id.

## Latency Budgets and SLAs

Edge termination: under 20 ms p95. Cache hit read: under 5 ms p95 regional. Cache miss read roundtrip: under 50 ms p95. Inventory write and confirm: under 120 ms p95 in-region. Leaderboard top-100 fetch: under 120 ms p95. Matchmaking placement: under 400 ms p95. Availability: read APIs 99.95 percent, writes 99.9 percent, realtime plane 99.9 percent.

## Key Algorithms and Workflows

Cache keys and versioning Use namespaced keys tenant colon feature colon entity id. Store a version or checksum alongside data. Invalidate by version bump via stream event.

Negative caching Cache known-empty for short TTL to suppress thundering herds on non-existent resources. Hot key mitigation Prefix randomization for counters, Redis client-side caching for replicas, and application-level sharding of celebrity keys. Leaderboards Use Redis ZSET for hot windows and a batched write-through to OLAP. For global boards, keep regional shards and compute federated top-N with min-heaps. Matchmaking QoS Policy ranks candidates by RTT, jitter, and load. Cache recent QoS probes at edge keyed by ASN to avoid network storms. Content delivery CDN with signed URLs and cache keys including locale, platform, and quality preset. Client-side caching with manifest-based versioning.

Resilience Read-through fallback to snapshots on partial outages. Circuit breakers around downstreams. Retries with jitter and idempotency on writes.

Security Reliability and Compliance Security mTLS in mesh; OAuth scopes and RBAC; WAF and bot mitigation; signed webhooks; input validation; SSRF egress policies. KMS-managed encryption for data at rest; per-tenant keys optional. Privacy Pseudonymous identifiers; minimize PII; DSAR APIs; retention controls by data class; residency routing per tenant. Reliability Multi-AZ per cell; stateless tiers active-active; stateful stores with quorum or replicas. Replay and backpressure with Kafka. Disaster recovery with tested restores and object store versioning. Change management Canary rollouts by shard subset and region; progressive traffic shifting; automatic rollback on SLO burn and error budgets.

## Multi-Tenancy

Tenant id in every key, topic, and row. Logical isolation with namespaces and quotas; optional dedicated cells and VPCs for premium tenants. Per-tenant encryption keys and dashboards; per-tenant cache partitions and budgets.

CI CD and Infrastructure as Code Everything as code Terraform or Pulumi and Helm. SAST DAST and SBOM with signed images. Argo Rollouts for canary; database or KV migrations with online DDL, dual writes, and shadow reads. Contract tests via schema registry for streams and APIs.

APIs and Integration Points Client GET profile, GET inventory, POST inventory mutate, GET leaderboard top, POST score submit, POST qos probe, GET shard route. Server POST match result, POST assignment, POST inventory grant, POST config publish. Admin POST reshard plan apply, POST cache purge, GET shard map, GET hot keys, GET error budget. Integrations Payments, identity, anti-cheat, push providers, analytics export.

Observability and Operational Practices Golden dashboards Cache hit ratio by feature, p95 and p99 per API, Redis ops per second, hot key distribution, shard load skew, reshard progress, Kafka lag, inventory write conflicts, leaderboard update tail. Tracing and logs OpenTelemetry traces; structured logs with tenant id and shard id; sampling tuned for hot paths. Autoscaling Scale by tail latency and queue depth. Pre-warm caches for events or drops. Predictive scaling from marketing calendars. Runbooks Cold cache bootstrap, hot key mitigation, reshard rollback, partial region isolation, snapshot restore, leaderboard rebuild.

Capacity Planning and Cost Estimate Illustrative busy region: 10 to 50k RPS reads to inventory, 2 to 10k RPS writes, leaderboard updates 10 to 100k per minute, realtime players 50k to 300k. Redis: 12 to 24 shards at under 65 percent memory. OLTP: Aurora writer plus 3 to 5 replicas or Scylla with RF=3. Kafka: 6 to 9 brokers with RF=3. Rough monthly cost per busy region: 30k to 120k depending on concurrency, egress, and retention.

## Testing Strategy and Failure Drills

Unit tests for cache keying, invalidation, and version math. Property tests for reshard correctness. Contract tests for APIs and streams. Load tests for cache churn and hot keys. Soak tests for memory fragmentation. Chaos drills for shard loss, cache evictions, broker lag, and regional isolation. Game days for event surges and reshard operations with rollback checkpoints.

## Phased Build Plan

MVP 3 months Single-region cell; Redis cache for profiles and inventories; leaderboard hot path; shard map service with static ranges; canary deployment; observability baseline. Phase 2 6 to 9 months Multi-region; dynamic reshard with dual write; QoS edge probes; negative caching; federation for global leaderboards; autoscaling. Phase 3 12 months Hard isolation per tenant, predictive cache warming,

automated reshard planners, warehouse-driven player segmentation for cache priming, stricter SLOs.

Key Choices Trade-Offs Risks and Mitigations

Cache staleness vs availability Favor bounded staleness with version keys and short TTLs; expose consistency hints to clients. Operational complexity vs performance Sharding improves scale but increases ops cost; mitigate with shard maps, automation, and strong observability. Hot keys and celebrity effects Mitigate with partitioned counters and tiered caches; enforce rate limits. Cross-shard transactions Avoid when possible; use sagas and idempotent compensations; provide eventual consistency guarantees.